

■ Forensic Entity-Action Profiler (FEAP) - Detailed Explanation

Here is a comprehensive, deep-dive explanation of your project, the **Forensic Entity-Action Profiler (FEAP)**, broken down by its architecture, libraries, and function-by-function behaviors.

1. Project Overview

FEAP is an advanced, Object-Oriented Natural Language Processing (NLP) dashboard built to decode weaponized language and "logistical ciphers" (hidden meanings). Specifically tailored for the Epstein email corpus, it uses the "Lemalock Hypothesis" to analyze pragmatics—finding out if seemingly innocent words (like *pizza* or *massage*) are actually functioning as coded terms for illicit logistics.

It does this through syntactic dependency parsing, entity graph mappings, topic modeling, time-series analysis, and zero-shot threat classification.

■ 2. Libraries and Tech Stack Used

The project rests heavily on state-of-the-art Python Data Science and NLP libraries.

- **Streamlit (streamlit):** The core UI framework handling the frontend rendering, interactivity, state management, and real-time visualization of the dashboard.
- **pandas & numpy:** Used heavily for data manipulation, particularly in calculating rolling-averages and metrics over time in the Diachronic Timeline module.
- **spaCy (spacy):** An industrial-strength NLP library used for dependency parsing (breaking a sentence down to subject/verb/object structures) and finding modified keywords.
- **NLTK (nltk):** Used for Named Entity Recognition (NER), chunking, and POS-tagging to locate exact Entities (People, Organizations, Locations) within the text logs.
- **NetworkX & Streamlit-Agraph (networkx, streamlit-agraph):** Graph logic mapping. It maps relational distances between different extracted entities, powering the "Spiderweb" visualization.
- **Gensim (gensim):** Used for **LDA (Latent Dirichlet Allocation)**. It handles unsupervised topic modeling to cluster words into hidden conversational themes.
- **Transformers & PyTorch (transformers, torch):** HuggingFace's transformers are used to run a Zero-Shot intent classification deep-learning model (cross-encoder/nli-MiniLM2-L6-H768) on CPU mode.

■■ 3. Backend Engine Breakdown (engine.py)

The backend is structured efficiently around a base class, `CorpusProcessor`, which all 5 specialized analytic modules inherit from.

The Wordlists (Dictionaries)

At the top of the file, four critical sets define the "cipher" dictionaries: `* FOOD_CODE_WORDS` & `* EUPHEMISM_WORDS`: Lists of high-risk coded words (e.g., pizza, massage, island). `* HEDGE_WORDS`: Modal verbs used to soften communication or build plausible deniability (e.g., might, perhaps, presumably). `* LOGISTIC_VERBS`: Verbs indicating movement or arrangement (e.g., arrange, deliver, transport).

Base Class: CorpusProcessor

This class handles all data loading and filtering without repeating code. * **load_clean()** & **load_annotated()**: Uses class-level caching (`_clean_cache`) so the large `.json` corpus datasets are only loaded into memory once. * **query_by_keyword()**: Uses Python's built-in `filter` and `lambda` functions to efficiently pull all emails containing the target search term. * **extract_sentences_with_keyword()**: Slices full email bodies into individual sentences (using `Regex`) and specifically extracts only the sentence containing the keyword. * **corpus_stats()**: Computes global metrics like total tokens and top unique senders.

Sub-Module 1: SyntacticXRay (spaCy)

- **Purpose:** To discover the grammatical context around the keyword.

analyse(): Loops through keyword-matched sentences and runs `spacy` to build a dependency tree.

- Finds the **ROOT** verb of the sentence. If the root verb is in `LOGISTIC_VERBS`, it gets flagged (e.g., "I will *deliver* the pizza").
- Walks the dependency tree to map syntactic edges (who is acting on what).
- Checks the sentence against the `HEDGE_WORDS` list to calculate "Modal hedging density".

Sub-Module 2: EntitySpiderweb (NLTK)

- **Purpose:** To track "who is talking to whom about what".
- **_extract_entities()**: Tokenizes text, assigns Parts-of-Speech, and chunks them to find Entities tagged as `PERSON`, `ORGANIZATION`, `GPE`, `LOCATION`, or `FACILITY`.
- **analyse()**: Iterates through emails, grabs all the entities, and pairs them together to form a "co-occurrence network" (e.g., if "John" and "Paris" are in the same email about "massage", the edge weight between them increases).

Sub-Module 3: ContextDecoder (Gensim LDA)

- **Purpose:** Prove statistically what an email is structurally about, bypassing the coded word.
- **_tokenize()**: Custom tokenizer stripping out stopwords (e.g., 'the', 'is', 'a').
- **analyse()**: Takes all emails hitting the keyword and turns them into a "Bag of Words" (`doc2bow`). It trains an `Unsupervised LdaModel` over 10 passes to generate `num_topics`. It outputs the hidden themes and assigns a dominant topic back to each specific email.

Sub-Module 4: DiachronicTimeline (Pandas)

- **Purpose:** Measure anomalies in language over time.
- **analyse():** Uses Pandas DataFrames to map every single email chronologically. It compares HEDGE_WORDS counts versus total word counts to get a hedge_density. It then applies a .rolling(10).mean() to smooth out the data and visually show spikes where the network suddenly starts hedging their language heavily—indicating pressure or panic.

Sub-Module 5: ThreatAssessor (Transformers)

- **Purpose:** Assign an "Intent Score" to communications.
- **get_pipeline():** Caches a Zero-Shot classification pipeline natively in PyTorch so it doesn't reload.
- **assess_email():** Feeds truncated emails to the AI model alongside specific labels (Logistical Coordination, Veiled Coercion, Social Grooming, etc.). The model returns a percentage score judging how likely the email reflects one of those illicit intents.

■ 4. Frontend UI Breakdown (app.py)

This ties everything together into a responsive, premium user interface.

- **Custom Theming:** A huge block of Custom CSS injects variables for a "Glassmorphism" light mode interface. It modifies exact UI elements (disabling Streamlit's default headers, expanding sidebars, creating custom Token Pills and Threat Badges).
- **Sidebar State:** Controls the keyword you are targeting. It features "Quick Targets" (pizza, massage, island) which uses st.session_state to immediately force a reload when clicked. Also includes sliders adjusting num_topics for Gensim.
- **Hero Dashboard Layout:** A globally computed Hero section shows total hits across the corpus for the targeted keyword.

5-Tab Navigation System: The app splits into 5 st.tabs mapping identically to the 5 engine.py modules.

- **Tab 1 (X-Ray):** Shows dependency relations, calculates the logistic vs hedging percentages via metric cards, and outputs custom Token Pills formatted to show the exact Part of Speech.
- **Tab 2 (Spiderweb):** Feeds the graph edges into streamlit_agraph for an interactive, draggable network node visualization. Nodes are color-coded based on Entity Type (Blue = Person, Red = Org, etc).
- **Tab 3 (Decoder):** Outputs the LDA clusters in customized flex-box card arrays, mapping confidence levels of different semantic themes.

- **Tab 4 (Timeline):** Displays `st . line_chart` showing off the rolling Pandas averages of language shifts. Extrapolates out peak threat timestamps.
- **Error Handling / Fallbacks:** If the user fails to install heavy libraries (like `spacy` or `gensim`), the UI gracefully throws an error but provides a regex/counter-based fallback to guarantee the dashboard still boots.